SIT315 - Transit Routing System

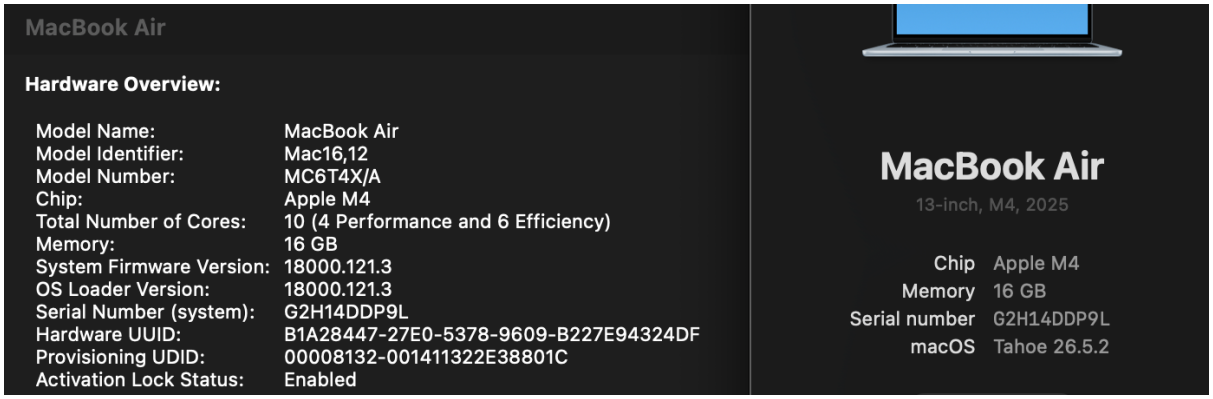
Mohammad Hassan Jamali – s225402184

Video Link: Link

GitHub Repository: https://github.com/hassan-jamali/vic_transit_routing

System Specifications

All implementations have been compiled and tested on a MacBook Air powered by an M4 chip (featuring 10 total CPU cores: 4 performance and 6 efficiency) with 16 GB of unified memory. The hardware specifications are detailed below.

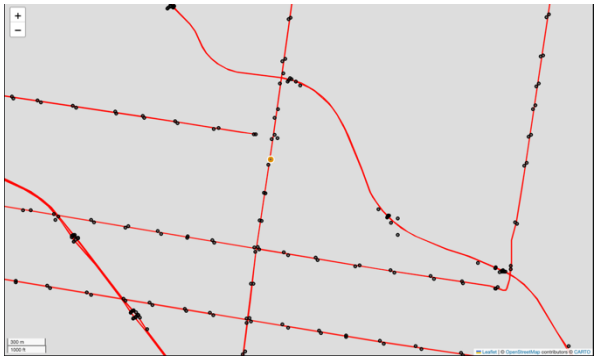


GTFS Data Architecture & Simulation

Firstly, the transit simulator relies on parsing real world General Transit Feed Specification (GTFS) data from the Victorian Transport Open Data Portal.

Since C++ is highly efficient at raw computation but less flexible with text parsing, the architecture splits into a Python data pipeline and a C++ routing system. In addition, the DataFile class utilises the pandas library to ingest stops.txt, stop_times.txt, and shapes.txt. Moreover, the Python wrapper pairs consecutive stops based on matching trip_id and sequential time boundaries to generate valid travel edges. Therefore, this data is exported into a lightweight transit_graph.json file, reserving the C++ TransitGraph class strictly for high-performance memory mapping and traversal.





Heuristic Design (Haversine Distance)

Firstly, the A* pathfinding algorithm relies on a heuristic function to estimate the remaining cost to the destination, preventing the algorithm from searching blindly. Since the transit network is mapped geographically, the simulator uses the Haversine formula in heuristic.cpp to calculate the great circle distance between two coordinates. In addition, by converting latitudes and longitudes into radians and applying the Earth's radius (6,371,000 meters), the heuristic provides a mathematically admissible straight-line distance.

heuristic.cpp > estimate_time(const TransitGraph &, const std::string &, const std::string &)

```cpp
#include "heuristic.h"
#include <cmath>

double Heuristic::calculate_distance(double latitude_one, double longitude_one, double latitude_two, double longitude_two) {
    // haversine distance in meters
    const double earth_radius_meters = 6371000.0;
    const double radians_per_degree = M_PI / 180.0;

    double delta_latitude = (latitude_two - latitude_one) * radians_per_degree;
    double delta_longitude = (longitude_two - longitude_one) * radians_per_degree;

    double haversine_a = std::sin(delta_latitude / 2) * std::sin(delta_latitude / 2) +
                         std::cos(latitude_one * radians_per_degree) * std::cos(latitude_two * radians_per_degree) *
                         std::sin(delta_longitude / 2) * std::sin(delta_longitude / 2);

    return earth_radius_meters * 2 * std::atan2(std::sqrt(haversine_a), std::sqrt(1 - haversine_a));
}

int Heuristic::estimate_time(const TransitGraph& graph, const std::string& current_id, const std::string& goal_id) {
    const auto& current_node = graph.nodes.at(current_id);
    const auto& goal_node = graph.nodes.at(goal_id);

    double distance_meters = calculate_distance(current_node.latitude, current_node.longitude, goal_node.latitude, goal_node.longitude);
    return static_cast<int>(distance_meters / 16.67);
}
```

Hybrid Parallel Architecture (Apple Metal)

Firstly, the heuristic distance calculations were reimplemented using Apple's Metal API to offload workloads from the CPU to the GPU. Since the architecture bridges C++ with Objective C++ (metal_heuristic.mm), it successfully compiles a Metal Shading Language (MSL) compute kernel (kBatchShaderSource) at runtime.

metal_heuristic.mm

```objc
#include "metal_heuristic.h"
#import <Metal/Metal.h>
#import <Foundation/Foundation.h>
#include <iostream>

namespace MetalHeuristic {

    // metal shader source embedded directly as a string
    static NSString* const kBatchShaderSource = @R"(
        #include <metal_stdlib>
        using namespace metal;

        kernel void calculate_haversine_batch(
            device const double* node_latitudes [[buffer(0)]],
            device const double* node_longitudes [[buffer(1)]],
            device const double* goal_latitudes [[buffer(2)]],
            device const double* goal_longitudes [[buffer(3)]],
            device double* heuristic_matrix [[buffer(4)]],
            constant uint& num_goals [[buffer(5)]],
            uint2 thread_position [[thread_position_in_grid]]
        ) {
            uint node_index = thread_position.x;
            uint goal_index = thread_position.y;

            double earth_radius_meters = 6371000.0;
            double degrees_to_radians = M_PI_F / 180.0;

            double latitude_1 = node_latitudes[node_index] * degrees_to_radians;
            double longitude_1 = node_longitudes[node_index] * degrees_to_radians;
            double latitude_2 = goal_latitudes[goal_index] * degrees_to_radians;
            double longitude_2 = goal_longitudes[goal_index] * degrees_to_radians;

            double delta_latitude = latitude_2 - latitude_1;
            double delta_longitude = longitude_2 - longitude_1;

            double a = sin(delta_latitude / 2.0) * sin(delta_latitude / 2.0) +
                       cos(latitude_1) * cos(latitude_2) *
                       sin(delta_longitude / 2.0) * sin(delta_longitude / 2.0);

            double c = 2.0 * atan2(sqrt(a), sqrt(1.0 - a));

            uint output_index = node_index * num_goals + goal_index;
            heuristic_matrix[output_index] = earth_radius_meters * c;
        }
    )";
```

In addition, the system allocates shared MTLBuffer memory blocks to pass array pointers for node and goal coordinates directly to the GPU.

```
ize_t node_buffer_size = node_count * sizeof(double);
ize_t goal_buffer_size = goal_count * sizeof(double);
ize_t matrix_buffer_size = node_count * goal_count * sizeof(double);

/ firstly, allocate shared memory buffers for nodes and goals
d<MTLBuffer> buffer_node_latitudes = [ctx.device newBufferWithBytes:node_latitudes.data() length:node_buffer_size options:MTLResourceStorageModeShared];
d<MTLBuffer> buffer_node_longitudes = [ctx.device newBufferWithBytes:node_longitudes.data() length:node_buffer_size options:MTLResourceStorageModeShared];
d<MTLBuffer> buffer_goal_latitudes = [ctx.device newBufferWithBytes:goal_latitudes.data() length:goal_buffer_size options:MTLResourceStorageModeShared];
d<MTLBuffer> buffer_goal_longitudes = [ctx.device newBufferWithBytes:goal_longitudes.data() length:goal_buffer_size options:MTLResourceStorageModeShared];
d<MTLBuffer> buffer_heuristic_matrix = [ctx.device newBufferWithLength:matrix_buffer_size options:MTLResourceStorageModeShared];
```

Moreover, a compute pipeline state is cached, allowing the GPU to spawn thousands of concurrent threads mapped via the [[thread_position_in_grid]] parameter. Therefore, once the command buffer commits via waitUntilCompleted, the engine immediately retrieves a fully populated matrix of distances.

Performance Profiling (Hybrid Architecture Evaluation)

Firstly, profiling was conducted to evaluate the trade-offs of the hybrid CPU / GPU architecture across varying workloads. In addition, single commuter routing was benchmarked across increasing route lengths and network disruptions (up to 2,000 dynamically blocked stations). Since single target A* pathfinding relies on a sequential min-heap priority queue, the CPU vastly outperformed the GPU, completing complex detours in 0.008 to 0.59 milliseconds while the GPU required around 52 to 55 milliseconds.

```
test_configs = [
    {"waypoints": 3,  "blocks": 1},
    {"waypoints": 5,  "blocks": 50},
    {"waypoints": 7,  "blocks": 250},
    {"waypoints": 10, "blocks": 500},
    {"waypoints": 15, "blocks": 1000},
    {"waypoints": 20, "blocks": 2000}
]

results = []

for config in test_configs:
    w = config["waypoints"]
    b = config["blocks"]

    # mute the print spam
    with contextlib.redirect_stdout(io.StringIO()):
        route, time_sec, primary_block = transit_data.get_valid_multi_route(num_waypoints=w, seed=10)

    blocks = [primary_block]

    if b > 1:
        all_stops = transit_data.stops['stop_id'].tolist()
        safe_stops = [stop for stop in all_stops if stop not in route]
        # ensure we don't try to block more stops than exist
        actual_blocks_to_add = min(b - 1, len(safe_stops))
        extra_blocks = random.sample(safe_stops, actual_blocks_to_add)
        blocks.extend(extra_blocks)

    command = ["--time", time_sec, "--route", *route, "--block", *blocks]

    results.append({
        "waypoints": w,
        "blocks": b,
        "cpu time (ms)": get_time(command, use_gpu=False),
        "gpu time (ms)": get_time(command, use_gpu=True)
    })

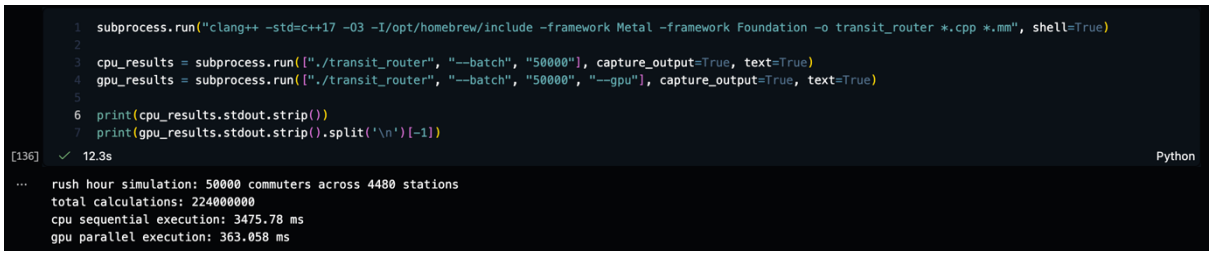
df = pd.DataFrame(results)
print(df.to_string(index=False))
```

[138] ✓ 24.3s Python

waypoints	blocks	cpu time (ms)	gpu time (ms)
3	1	0.234917	236.985417
5	50	0.467083	52.265166
7	250	0.460708	58.554792
10	500	0.685167	53.211667
15	1000	0.569167	56.946917
20	2000	0.008417	55.777292

Moreover, this latency discrepancy is driven by macOS Metal driver overhead, which incurs costs for pipeline synchronisation (waitUntilCompleted) and shared buffer allocation on every independent query. Therefore, this demonstrates Amdahl's Law in practice: sequential algorithms with high synchronisation requirements cannot exploit SIMD/GPU parallelism and execute significantly faster within the CPU's local L1/L2 cache.

In addition, when shifting from sequential graph traversal (AStarRouter::find_route) to batch heuristic precalculation (simulating 50,000 commuters across 4,480 stations), the workload converted into a fully decoupled, parallel matrix operation. Since the GPU bypassed sequential branching to run 224 million calculations across thousands of concurrent threads independently, the Apple Metal kernel (calculate_batch_distances) completed the batch in 338.2 milliseconds compared to the CPU's 3,198.8 milliseconds. Therefore, the hybrid pipeline successfully demonstrated nearly a 10x throughput advantage once thread saturation masked driver overhead.



Stress Test

Firstly, an extreme stress test was configured by ingesting all 6 GTFS transport modes (Regional Train, Metropolitan Train, Metropolitan Tram, Bus, and Coach) combined with a 50,000 commuter batch request to determine system breaking points. In addition, loading all 6 modes via DataFile(transport_modes=[1, 2, 3, 4, 5, 6]) scaled the transit graph from 4,480 stations to tens of thousands of combined stops and transit edges.

Since the Metal compute kernel constructs a dense Haversine distance matrix of dimensions $\text{node_count} \times \text{commuter_count}$, the required memory allocation scaled to:

$$\text{matrix_buffer_size} = \text{node_count} \times 50{,}000 \times 8 \text{ bytes}$$

Moreover, on a graph exceeding 30,000 stops, this single contiguous matrix required over 12 GB of uncompressed floating-point memory, operating alongside graph adjacency structures within the 16 GB unified memory architecture. Since [ctx.device newBufferWithLength:options:] was unable to allocate the requested contiguous buffer under macOS memory pressure, the allocation returned nil, triggering an unhandled memory fault and an immediate crash, recorded as a Did Not Finish (DNF).